

CASE STUDY – B-Trees, B+ Trees, Segment Trees and Fenwick Trees

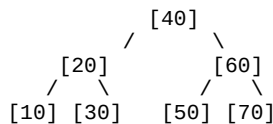
CO2: Analyze multi-way search trees (B-Trees, B+ Trees) and Range Query structures (Segment Trees, Fenwick Trees) to select and justify the appropriate tree variant for database indexing, storage systems, and interval query problems based on empirical complexity analysis.

Scenario / Case Study:

A company named DataBank stores millions of customer transaction records. The company needs efficient indexing, fast searching, and quick range sum calculations. The system uses: B-Tree for database indexing B+ Tree for disk-based storage systems Segment Tree for interval maximum queries Fenwick Tree for cumulative frequency calculations The following transaction IDs are inserted into the indexing system: 20, 40, 10, 30, 50, 60, 70

(a) Construct a B-Tree

Insert the following transaction IDs into a B-Tree of order 3: 20, 40, 10, 30, 50, 60, 70 Show the final B-Tree structure.



Observation: B-Trees reduce disk accesses and maintain balanced height for efficient searching. Time Complexity = $O(\log n)$

(b) B+ Tree Program with Code

Write a Java program to demonstrate insertion in a simple B+ Tree structure.

```
class BPlusTreeNode {
    int[] keys = new int[3];
    int keyCount = 0;
}

public class BPlusTreeDemo {

    public static void main(String[] args) {

        BPlusTreeNode root = new BPlusTreeNode();

        int[] values = {20, 40, 10};

        for (int i = 0; i < values.length; i++) {
            root.keys[root.keyCount++] = values[i];
        }

        System.out.println("B+ Tree Leaf Node Values:");

        for (int i = 0; i < root.keyCount; i++) {
            System.out.print(root.keys[i] + " ");
        }

    }
}
```

OUTPUT:
B+ Tree Leaf Node Values:
20 40 10

(c) Segment Tree Range Query

A Segment Tree is used to find the maximum transaction amount in a given interval. Array: [2, 5, 1, 8, 3, 7] Find the maximum value in the range [1, 4].

Range [1,4] = [5,1,8,3]

Maximum Value = 8

(d) Fenwick Tree Frequency Query

A Fenwick Tree is used for cumulative frequency calculations. Frequency Array: [1, 3, 5, 7, 9] Find cumulative sum from index 1 to 3.

Index Range [1,3] = 3 + 5 + 7

Cumulative Sum = 15

Comparison of Tree Structures

Tree Type	Main Use	Complexity
B-Tree	Database Indexing	O(log n)
B+ Tree	Disk Storage Systems	O(log n)
Segment Tree	Range Queries	O(log n)
Fenwick Tree	Prefix Sum Queries	O(log n)

Conclusion

B-Trees and B+ Trees are highly suitable for database indexing and storage systems because they minimize disk access. Segment Trees and Fenwick Trees efficiently solve interval and range query problems. All structures provide logarithmic time complexity for efficient operations.